TEMA 2

Instalación y uso de IDEs

Preparación de Eclipse y flujo de trabajo profesional con Java

Propósito del tema

Comprender el proceso, utilizar las herramientas con criterio y documentar decisiones reproducibles.

ENTORNOS DE DESARROLLO · 1.º DAM

Qué aprenderás

- Explicar qué integra un IDE y qué ventajas aporta.
- Instalar y configurar JDK y Eclipse de forma reproducible.
- Crear, organizar, compilar y ejecutar proyectos Java.
- Utilizar ayudas de edición, depuración, complementos y actualizaciones.

> **Ruta de aprendizaje**
> Conceptos → decisiones técnicas → ejemplos en Java → comprobación y documentación.

Índice del tema

1. Qué es un IDE
2. Del editor aislado al IDE
3. Requisitos previos: JDK y arquitectura
4. Instalación de Eclipse
5. Workspace, perspectivas y vistas
6. Crear un proyecto Java
7. Editar, compilar y ejecutar
8. Ayudas de edición y navegación
9. Refactorización segura
10. Depuración paso a paso
11. Complementos y Marketplace
12. Actualización y mantenimiento del IDE
13. Control de versiones y copias
14. Optimización y resolución de problemas

1. Qué es un IDE

Un entorno de desarrollo integrado, o IDE, reúne en una aplicación las herramientas necesarias para escribir, construir, ejecutar, probar, depurar y mantener software. La integración reduce cambios de contexto y conserva una visión común del proyecto.

Componente	Función principal
Editor de código	Resaltado, completado, formato, búsqueda y navegación
Compilador o intérprete	Traducción y detección de errores
Constructor	Automatización de compilación y empaquetado
Depurador	Pausa, avance controlado y observación del estado
Pruebas	Ejecución y presentación de resultados
Control de versiones	Historial y colaboración

Ventaja clave

El IDE no sustituye la comprensión del proceso: la hace más rápida, visible y repetible.

2. Del editor aislado al IDE

Sin IDE, una aplicación Java se guarda como archivo .java, se compila con javac y se ejecuta con java. Comprender esta secuencia ayuda a interpretar lo que Eclipse automatiza.

```
javac HolaMundo.java
java HolaMundo
```

Eclipse invoca el compilador del JDK, mantiene una estructura de proyecto, muestra los diagnósticos y ejecuta la clase que contiene main. Los mismos conceptos siguen existiendo aunque estén detrás de botones y menús.

Diagnóstico útil

Si un programa funciona desde el IDE pero no desde la terminal, revisa la versión de Java, el classpath, el directorio actual y la configuración de ejecución.

3. Requisitos previos: JDK y arquitectura

Para desarrollar en Java se necesita un JDK, no solo un entorno de ejecución. Conviene utilizar una versión LTS compatible con el curso y la misma arquitectura que el sistema operativo y el IDE.

- Comprobar la versión instalada con java -version y javac -version.
- Evitar varias instalaciones sin documentar cuál utiliza Eclipse.

- Guardar instaladores y anotar versión, proveedor y fecha de actualización.
- Mantener separadas las carpetas de programas, proyectos y copias de seguridad.

```
java -version
javac -version
```

4. Instalación de Eclipse

El instalador oficial permite elegir una distribución. Para DAM se selecciona Eclipse IDE for Java Developers, se indica la JVM y la carpeta de instalación, se revisa la licencia y se inicia el entorno.

Paso	Comprobación
1. Descargar	Origen oficial y versión adecuada al sistema
2. Seleccionar paquete	Eclipse IDE for Java Developers
3. Elegir JVM	JDK del curso, no una versión accidental
4. Instalar	Ruta conocida y permisos suficientes
5. Iniciar	Workspace de aula y pantalla principal sin errores

Guía para el alumnado

Incluye capturas de cada pantalla, la versión exacta y una prueba final con HolaMundo. Una guía debe permitir detectar en qué paso se ha producido un fallo.

5. Workspace, perspectivas y vistas

El workspace es la carpeta de trabajo que contiene proyectos y metadatos de Eclipse. Una perspectiva distribuye vistas para una tarea; las vistas muestran proyectos, problemas, consola, estructura o depuración.

Elemento	Qué representa	Recomendación
Workspace	Área de trabajo del curso	Una ruta corta, estable y con copia de seguridad
Perspective	Distribución de herramientas	Java para programar; Debug para depurar
Package Explorer	Jerarquía de proyectos y paquetes	Navegar, no editar archivos fuera del IDE
Problems	Errores y avisos	Corregir primero el diagnóstico inicial
Console	Entrada y salida del programa	Distinguir consola terminada de consola activa

6. Crear un proyecto Java

Un proyecto agrupa código, configuración y resultados de compilación. En Eclipse se crea con File → New → Java Project. Después se añaden paquetes y clases dentro de src.

```
MiProyecto/
├─ src/
│  └─ es.centro.dam/
│     └─ HolaMundo.java
├─ bin/
│  └─ es/centro/dam/HolaMundo.class
└─ .project y .classpath
```

- Los paquetes se escriben en minúsculas y evitan el paquete por defecto.
- El nombre de una clase pública coincide con el archivo .java.
- src contiene fuentes; bin contiene bytecode generado y no se edita a mano.

7. Editar, compilar y ejecutar

```
package es.centro.dam;

public class HolaMundo {
    public static void main(String[] args) {
        System.out.println("Hola, DAM");
    }
}
```

Con Build Automatically activo, Eclipse recompila al guardar. Para ejecutar, se selecciona la clase con main y se utiliza Run As → Java Application. La salida aparece en Console y los errores en Problems.

Señal	Interpretación	Acción
Subrayado rojo	Error que impide compilar	Leer mensaje y corregir la causa
Subrayado amarillo	Advertencia o mejora posible	Comprenderla antes de ignorarla
Consola sin salida	Puede ejecutarse otra configuración	Revisar clase principal y consola seleccionada

8. Ayudas de edición y navegación

El IDE aumenta la productividad mediante completado, correcciones rápidas, navegación semántica y búsqueda. Deben usarse como ayudas conscientes, no como sustituto de la lectura.

- Content Assist propone miembros, tipos y plantillas según el contexto.
- Quick Fix ofrece correcciones para diagnósticos concretos.
- Open Declaration permite saltar a la definición de un elemento.
- References localiza dónde se utiliza una clase, método o variable.
- Format Source aplica una presentación homogénea.

Atajo mental

Antes de aceptar una propuesta automática, predice qué cambio realizará y comprueba el diff o el resultado.

9. Refactorización segura

Refactorizar modifica la estructura interna sin cambiar el comportamiento observable. El IDE puede renombrar, extraer métodos, mover tipos o cambiar firmas actualizando referencias de manera coordinada.

Refactorización	Cuándo usarla	Control posterior
Rename	Un nombre no comunica intención	Buscar referencias y ejecutar pruebas
Extract Method	Un bloque realiza una tarea identificable	Comprobar parámetros y retorno
Move	Una clase está en un paquete inadecuado	Revisar imports y visibilidad
Change Method Signature	Cambian parámetros o tipo de retorno	Revisar todos los llamadores

Condición de seguridad

Una refactorización necesita pruebas o, como mínimo, casos de comprobación antes y después del cambio.

10. Depuración paso a paso

El depurador permite detener el programa en un punto de ruptura y observar variables, expresiones, pila de llamadas e hilos. Sirve para contrastar una hipótesis sobre el fallo.

Acción	Efecto
Breakpoint	Pausa antes de ejecutar una línea
Step Over	Ejecuta la línea sin entrar en métodos llamados

Acción	Efecto
Step Into	Entra en el método invocado
Step Return	Termina el método actual y vuelve al llamador
Resume	Continúa hasta el siguiente punto de ruptura
Variables / Expressions	Muestra estado y evalúa expresiones

```
int total = 0;              // breakpoint
for (int n : numeros) {
    total += n;              // observar n y total
}
double media = (double) total / numeros.length;
```

Método de depuración

Reproduce → formula una hipótesis → pausa antes del desvío → observa → corrige un cambio pequeño → vuelve a probar.

11. Complementos y Marketplace

Los complementos amplían el IDE con herramientas de diseño, análisis, lenguajes o integración. En Eclipse se gestionan desde Eclipse Marketplace o Install New Software.

- Instalar solo desde fuentes fiables y comprobar compatibilidad de versión.
- Añadir el mínimo de componentes necesarios para reducir conflictos y consumo.
- Documentar complementos obligatorios para que el entorno sea reproducible.
- Reiniciar cuando la instalación o actualización lo requiera.

Ejemplos del material

WindowBuilder permite diseñar interfaces Swing visualmente; Papyrus incorpora modelado UML. Su instalación debe responder a una necesidad didáctica concreta.

12. Actualización y mantenimiento del IDE

Actualizar corrige vulnerabilidades y defectos, mejora compatibilidad y añade funciones. Sin embargo, una actualización puede afectar complementos o proyectos, por lo que conviene planificarla.

Antes	Durante	Después
Guardar proyectos y anotar versiones	Leer cambios y seleccionar componentes	Reiniciar, compilar y ejecutar un proyecto de prueba
Comprobar compatibilidad de plugins	No interrumpir el proceso	Verificar JDK, workspace y configuraciones

Aula estable

Congela versiones durante una unidad didáctica y prueba las actualizaciones en un equipo antes de extenderlas a toda la clase.

13. Control de versiones y copias

Git registra cambios del proyecto y permite colaborar. No sustituye a una copia de seguridad del equipo, pero aporta historial, comparación y recuperación a nivel de código.

```
Proyecto Eclipse → repositorio Git → commits pequeños → remoto
           workspace y bin no deben confundirse con el historial
```

- Versionar fuentes y archivos de configuración necesarios.
- Excluir resultados generados como bin cuando el proyecto puede reconstruirlos.
- Escribir mensajes de commit que expliquen el cambio y su finalidad.
- Comprobar que el proyecto puede importarse y ejecutarse en otro equipo.

14. Optimización y resolución de problemas

Optimizar el entorno significa reducir esperas sin comprometer estabilidad. Antes de cambiar memoria, plugins o compilación, se mide el síntoma y se identifica el cuello de botella.

Síntoma	Comprobaciones
Eclipse no inicia	JDK compatible, ruta, permisos y archivo de registro
Proyecto con errores tras importar	JRE System Library, nivel del compilador, dependencias y codificación
IDE lento	Número de proyectos abiertos, plugins, memoria, indexación y antivirus
Clase no ejecutable	Método main, paquete, errores de compilación y Run Configuration

Principio de optimización

Primero medir, después cambiar una sola variable y finalmente verificar. En este tema, todo el contenido técnico se presenta en español.

Síntesis y autoevaluación

- Un IDE integra edición, construcción, ejecución, depuración, pruebas y versiones.
- Un entorno reproducible exige documentar JDK, Eclipse, workspace y complementos.
- src contiene fuentes y bin resultados generados; paquetes y clases organizan el proyecto.
- Depurar implica observar evidencia y verificar hipótesis, no modificar al azar.

Reto final · Comprueba que dominas el tema

Redacta una guía de instalación de Eclipse para el aula: requisitos, descarga, selección del JDK, workspace, creación de HolaMundo, ejecución, captura de errores comunes y lista final de comprobación.

Material base

Tema reorganizado, ampliado y redactado en español a partir de entornos-instalación-y-uso-de-ides.pdf, aportado por el docente. No se reproducen los ejercicios del documento original.